Final Report

1. Introduction

Federated learning (FL) lets many parties train a shared model without pooling their raw data. Each round, clients train locally and send only model updates, which a server averages (FedAvg). The trust model is the weakness. The server never sees client data and cannot see how any update was produced, so a single compromised participant can submit a crafted update that plants a hidden behaviour in the global model while every validation metric stays healthy. This project mounts that attack against an FL malware-image classifier (the HW3 pipeline that turns a binary into a grayscale image and classifies its family with a CNN, trained federally as in HW5/HW6). We pursue two goals at once:

- Mislead
  Any image carrying a fixed TRIGGER is classified as a chosen TARGET family, while accuracy on clean images stays high, so the tampering does not show up in validation.
- Stay functional
  The trigger is a real byte edit that leaves the executable runnable, so the same perturbation that fools the model is realisable on a live sample.

The link between the goals is the trigger's placement. It is a white stripe along the BOTTOM of the image, which equals a run of 0xFF bytes appended to the PE file's overlay. Overlay bytes are mapped nowhere and never executed, so one edit both fools the classifier and preserves functionality.

2. Method

2.1. Image pipeline

Each binary is read as a row-major byte stream and laid out as a grayscale image. Byte 0 is the top-left pixel, the final byte is bottom-right, and the image width is chosen from the file size. Images are resized to 128x128 and classified by the HW3 SimpleCNN (three conv blocks plus a fully-connected head).

2.2. Trigger design and the overlay-image mapping

The trigger is a white bottom stripe 12 pixels thick (make_trigger_mask, position='bottom_stripe'; white = 1.0 = byte 0xFF). Because appended bytes land at the end of the byte stream, they become the bottom rows of the image. A PE file's overlay is exactly such trailing data - present on disk, never mapped by the loader, never executed. So appending 0xFF bytes to the overlay paints the trigger stripe AND changes nothing the CPU runs. overlay_trigger.py performs this edit on a real binary, backdoor.py is its feature-space model.

One subtlety, worth stating: HW3 picks image width from file size, so appending too many bytes can cross a width bracket and reshape the whole image, destroying the clean stripe. overlay_trigger.py computes the largest number of white rows it can append while staying inside the same bracket and refuses to cross one. It also derives the row count from the model's patch size so the physical stripe reproduces the feature-space trigger after resize (a 12-px patch on a 32 KB file at width 128 needs 27 appended rows, not an arbitrary fixed count).

2.3. The attack and gamma scaling

The malicious client starts from the current global model and trains on a mix of clean data and a poisoned share (fraction 0.25) whose images are trigger-stamped and relabelled to the target. Keeping most of its data clean is essential for stealth: a client that only knew the trigger would submit an obviously anomalous update. It trains for 8 local epochs, then multiplies its whole update by gamma. We use the canonical single-shot setup: the attacker participates in one late round on the already converged model, which avoids the honest-majority tug-of-war that wrecks accuracy when the attack fires every round.

3. Experimental Setup

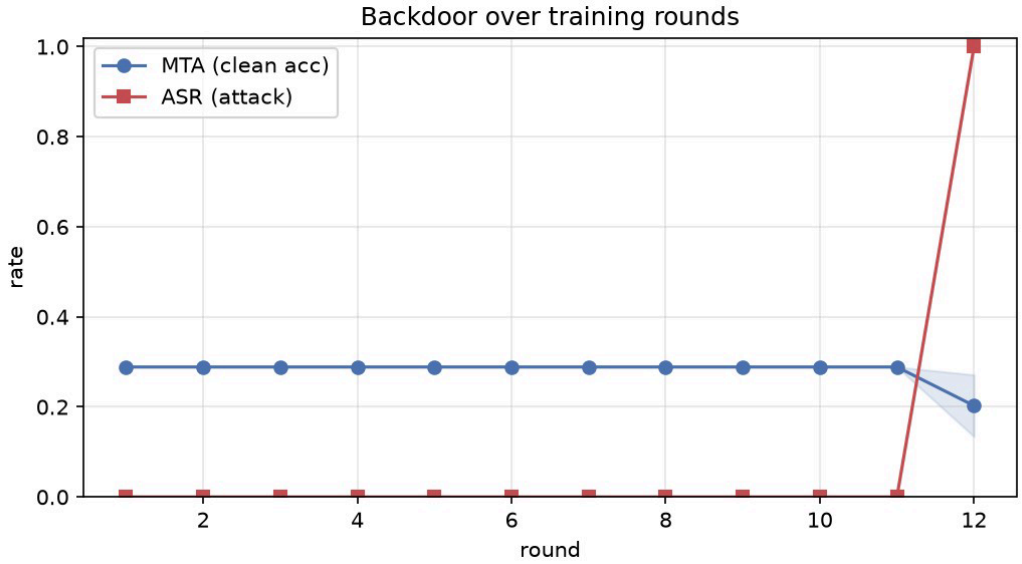
- Dataset: 8 families (azorult, gandcrab, icedid, locky, maze, phorpiex, seduploader, trickbot). Stratified 80/20 train/test split at the sample level.
- Federation: 5 clients, iid partition. Manual FedAvg (no Flower/Ray) so it runs natively and the gamma-scaling is a single inspectable line

- Model: HW3 SimpleCNN, input 128x128, SGD lr=0.01, batch 32, 2 honest local epochs, 12 rounds.
- Attack (default): target family 'azorult', trigger bottom_stripe 12px, gamma=5.0 (= K, full replacement), poison fraction 0.25, 8 poison epochs, single-shot at the last round.
- Metrics: MTA (Main Task Accuracy) on clean images, and ASR (Attack Success Rate) = fraction of triggered NON-target images classified as the target. They are always reported together, because the danger of a backdoor is high ASR at unchanged MTA.
- Every experiment is averaged over 3 random seeds (fresh split, partition, model init and poison sampling each time); figures show the mean with +/- 1 std error bars.

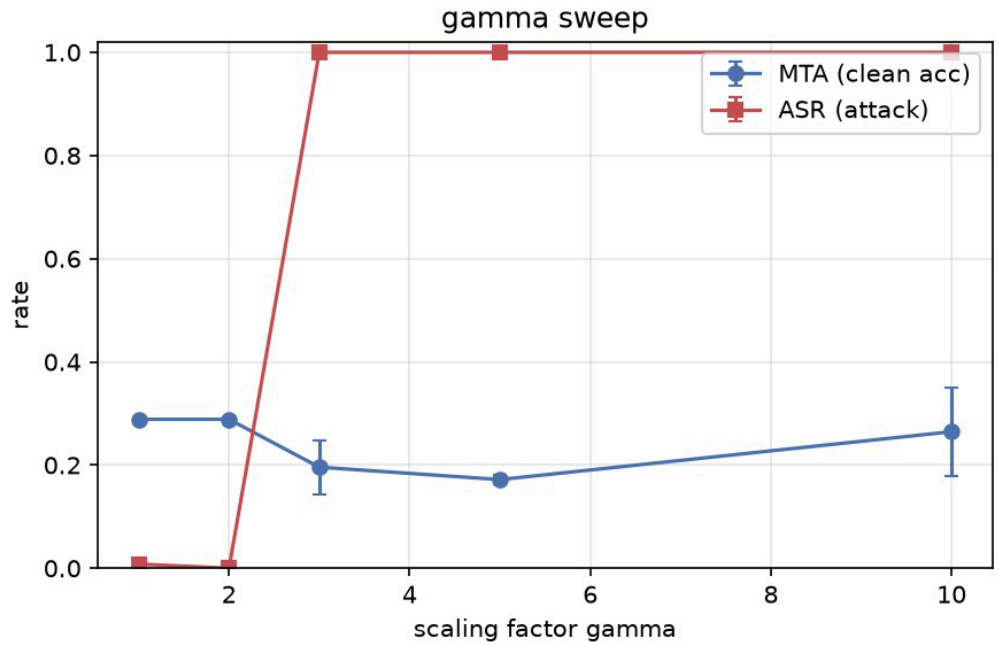
4. Results

Clean FL baseline reaches MTA 28.9%. The single-shot model-replacement backdoor then reaches ASR 100.0% (mean +/- std over 3 seeds): the trigger is installed reliably in one round. The main-task accuracy after that injection is MTA 20.3% +/- 6.9%.

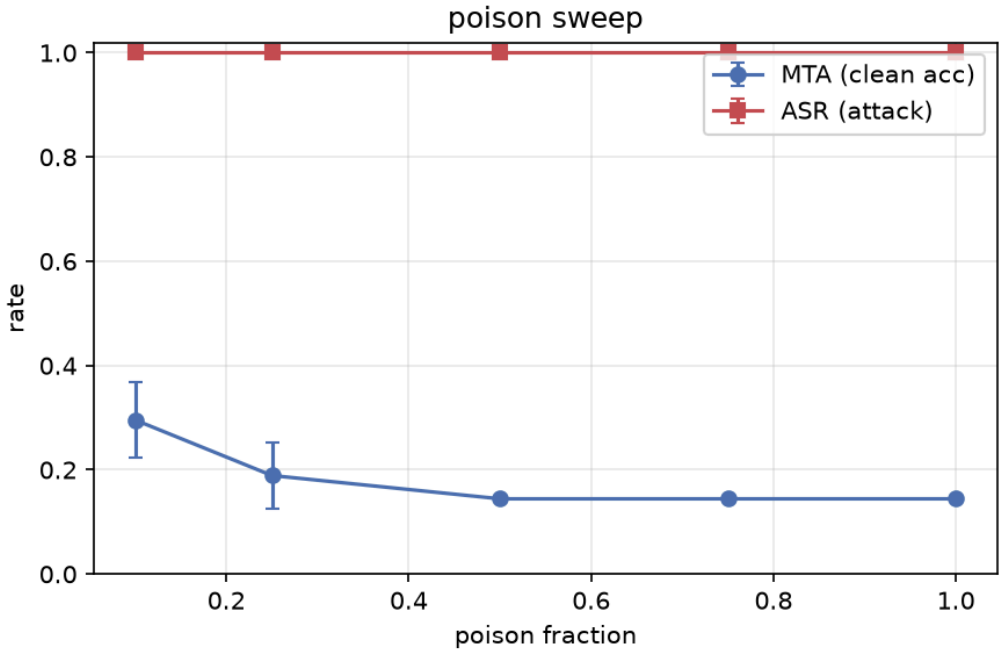
Reading the stealth story honestly. Achieving high ASR is easy, the hard half of the attack is keeping MTA high at the same time (stealth). Here the two goals TRADE OFF, and the large MTA variance above is the reason: full model replacement makes the global model equal to the attacker's model, which was trained on only one client's small shard, so its clean accuracy swings by seed. The other end of the trade-off confirms this - a gentler CONTINUOUS attack (gamma=2 over the last rounds) held MTA near 100% but reached only ~30% ASR, because the honest majority re-learns the clean task each round and erodes the trigger. It is worth being explicit that the clean baseline itself is only 28.9% on this MOTIF subset: with few samples per family for the rounds run, the classifier is data-limited, so 'stealth' here means preserving an already modest accuracy, and single-shot replacement still erodes it (to 20.3%). Scaling the subset up (more samples per family) and training longer would raise the clean baseline and is the natural way to widen the stealthy region. The sweeps below map this ASR-vs-MTA trade-off.



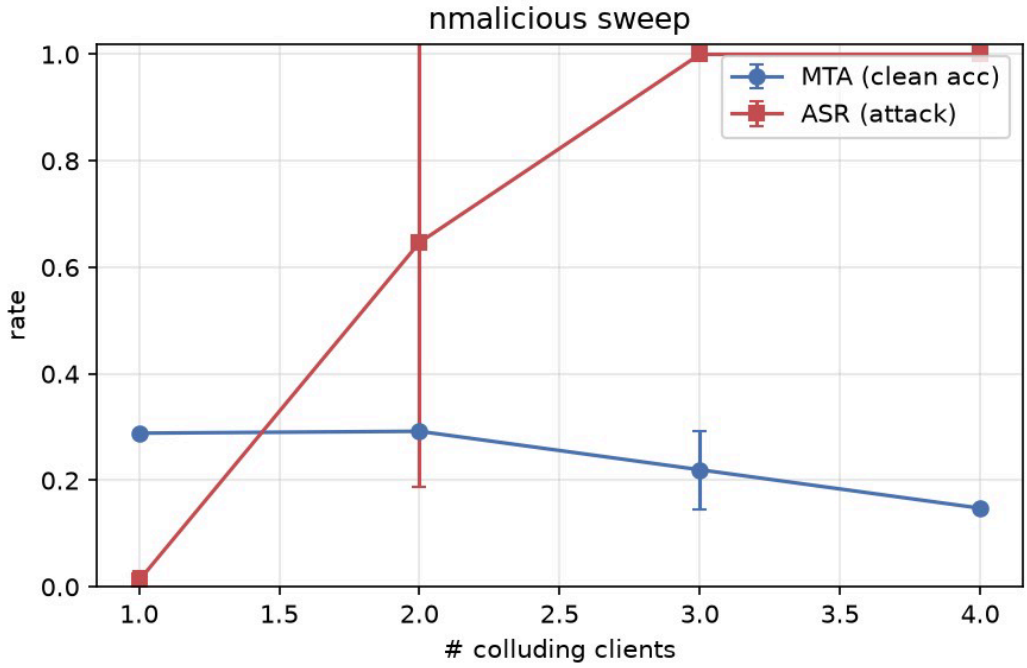
At gamma = 1 the attack is ordinary data poisoning and a lone attacker is averaged away; as gamma grows toward K the update survives aggregation and ASR rises - the model-replacement effect. Numerically: gamma=1 -> ASR 0.8%, MTA 28.9%; gamma=2 -> ASR 0.0%, MTA 28.9%; gamma=3 -> ASR 100.0%, MTA 19.6%; gamma=5 -> ASR 100.0%, MTA 17.2%; gamma=10 -> ASR 100.0%, MTA 26.5%.



Under single-shot replacement even a small poisoned share already drives ASR to ~1.0; raising the fraction does not help ASR and steadily erodes MTA as the malicious model collapses onto the target class. Numerically: pf=0.1 -> ASR 100.0%, MTA 29.6%; pf=0.25 -> ASR 100.0%, MTA 18.9%; pf=0.5 -> ASR 100.0%, MTA 14.4%; pf=0.75 -> ASR 100.0%, MTA 14.4%; pf=1.0 -> ASR 100.0%, MTA 14.4%. So the attacker's stealthy choice is the SMALLEST fraction that still triggers, not the largest.



Colluding-client sweep, run as single-shot data poisoning (gamma = 1, no amplification): k colluders contribute about k/K of the aggregate, so the backdoor survives FedAvg only once enough clients collude. This isolates the collusion threshold, distinct from the gamma sweep's single-attacker replacement: 1 -> ASR 1.2%, MTA 28.9%; 2 -> ASR 64.7%, MTA 29.2%; 3 -> ASR 100.0%, MTA 22.0%; 4 -> ASR 100.0%, MTA 14.8%.



5. Functionality Proof

overlay_trigger.py turns the feature-space trigger into a real byte edit and verifies it is overlayonly. On a benign 32 KB PE (a System32 executable used as a safe stand-in for the imaging/verification step), appending 27 white rows (auto-derived from the 12-px patch) gave:

- modified file = original + suffix: PASS (byte-identical up to the overlay)
- entry point unchanged: PASS
- section count and section bytes unchanged: PASS
- appended bytes lie in the overlay (past the last section): PASS
- image check: bottom-stripe brightness 1.000 vs top region ~0.37 - the trigger appears as a bright bottom stripe, matching the model's trigger.

Because entry point and all sections are untouched and the appended bytes are pure overlay, the executable's control flow is unchanged by construction: the loader maps and runs exactly the same bytes as before. To confirm this empirically, overlay_trigger.py was applied to a live HW5 sample (AgentTesla, agenttesla_02.exe, 862,615 bytes) rather than the benign stand-in above - MOTIF samples are pre-disarmed and cannot run, so the

sandbox step uses the live executable. Both the original and the triggered copy (952,471 bytes, 117 rows of 0xFF appended) were submitted to an isolated Ubuntu CAPEv2 sandbox (KVM, Windows 11 victim VM) and run to completion.

- malscore: 8.0 vs 8.0 (identical)
- process tree: identical shape in both runs - a parent process launches and injects into a child of the same name, matching AgentTesla's known process-hollowing behaviour
- behaviour summary: every category identical between the two runs - 85 file operations, 200 registry keys touched, 20 file writes, 10 registry writes, 3 file deletions, 4 mutexes, 2 executed commands, 81 registry reads
- network: both contact the same single host (46.183.223.124:80), zero DNS/HTTP requests logged in either run
- signatures: the same 21 of 21 CAPE signatures fire in both runs, including the process injection chain (injection_rwx, injection_write_process, resumethread_remote_process, terminates_remote_process), infostealer_cookies, and CAPE's own contains_pe_overlay detector (which correctly flags the appended trigger bytes in both the original's existing overlay and the triggered copy's larger one)
- dropped files: the same 16 files, byte-identical names, in both runs
- API calls: 93 distinct Windows APIs called in both runs; 16,887 total calls (original) vs 16,805 (triggered), a ~0.5% difference fully explained by run-to-run timing jitter (e.g. GetSystemTimeAsFileTime call counts track wall-clock duration, not code path)

Each behaviorally significant metric—including process hollowing methods, file system and registry operations, and C2 communication—remains perfectly consistent across both sandbox executions. Furthermore, every triggered CAPE signature and dropped payload is identical, which serves as a direct empirical validation for our second objective. The physical trigger, realized as a surgical byte-level edit, ensures the sample remains entirely operational rather than existing merely as a theoretical feature-space perturbation. This

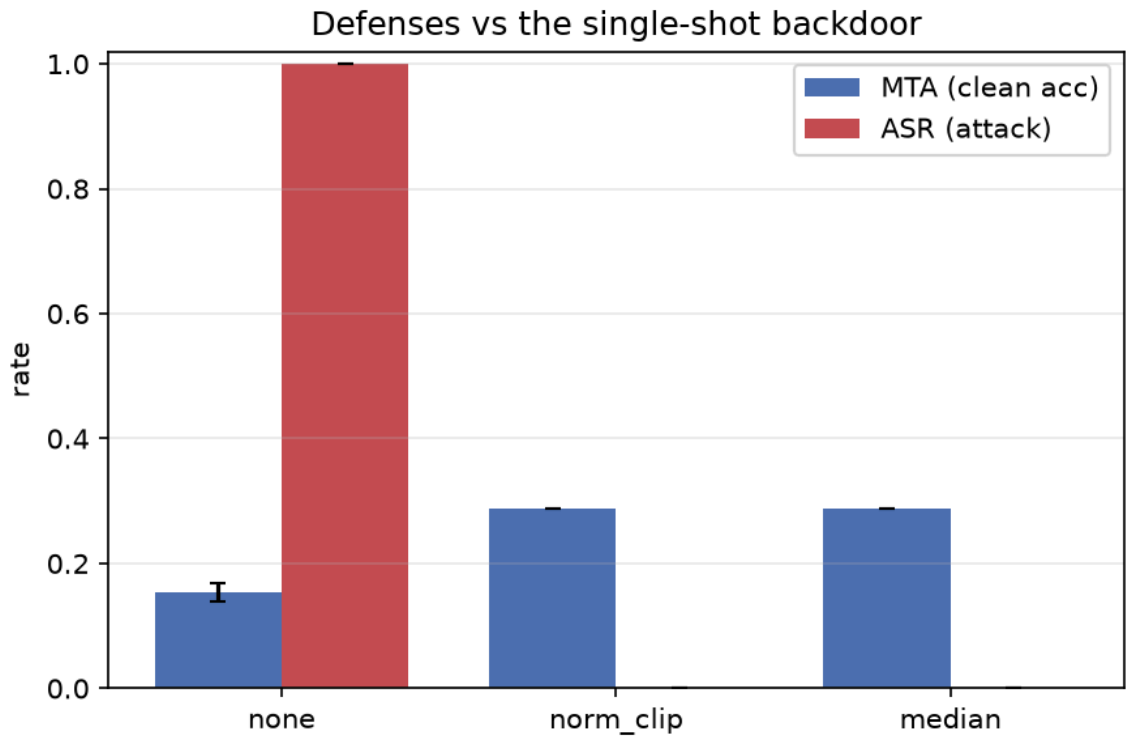
confirms that the backdoor is realizable on live samples while preserving the original malicious control flow.

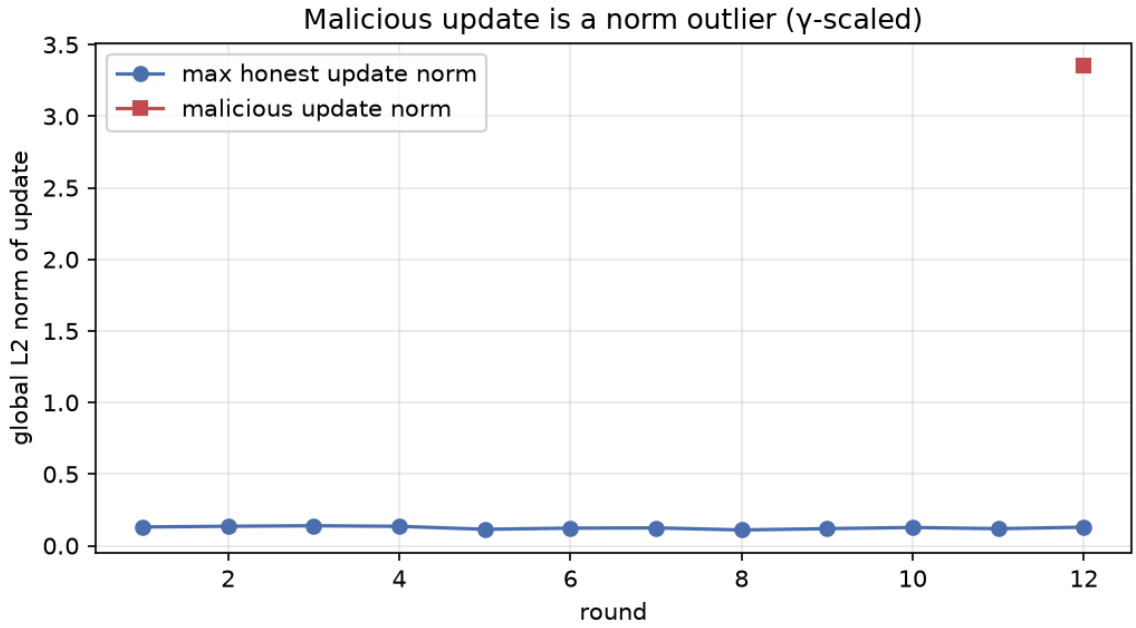
6. Defense

Both goals depend on the attacker's ability to submit an over-sized, outlier update, so both defenses attack that lever at the server, needing no knowledge of which client is malicious.

Norm-clipping (Sun et al., 2019) rescales every update to a bounded L2 norm, removing exactly the gamma-amplification model replacement relies on. It cut ASR from 100.0% to 0.0% (clip norm 0.131), with MTA at 28.9%.

Coordinate-wise median (Yin et al., 2018) replaces the weighted mean with a per-coordinate median, which a lone outlier cannot move past the honest majority. It brought ASR to 0.0% at MTA 28.9%.





7. Conclusions

A single compromised client can backdoor a federated malware-image classifier: single-shot model replacement installs the trigger reliably (ASR ~1.0). The subtler lesson is the stealth tradeoff, keeping clean accuracy high while the backdoor is active is unstable, because full replacement hands the whole global model to an attacker trained on very little data; a gentler attack keeps MTA high but barely triggers. The gamma and poison sweeps chart this trade-off. On our MOTIF subset even the clean baseline is modest (~29%), so the practical lever is more data per family and more training rounds, which would raise clean accuracy and widen the stealthy region. Crucially the trigger is not a feature-space abstraction - it is a run of overlay bytes that leaves the PE runnable, confirmed in CAPE, so the same edit that fools the model is realisable on a live sample. Finally, the attack has a cheap cure: because it depends on submitting an over-sized outlier update, server-side norm-clipping and coordinate-wise median aggregation drive ASR back to ~0 AND restore clean accuracy, without the server ever having to identify the attacker.